

CSE 331: Computer Security

Fundamentals

Spring 2026

Heap

Amir Rahmati
Xinrui Yu



Stony Brook Institute
at Anhui University

安徽大學紐約石溪學院

Back to memory errors?

- We have talked about memory quite a bit already...
 - .text: where the code lives
 - .plt: where library function stubs live
 - .got: where pointers to imported symbols live
 - .bss: used for uninitialized global writable data (such as global arrays without initial values)
 - .data: used for pre-initialized global writable data (such as global arrays with initial values)
 - .rodata: used for global read-only data (such as string constants)
 - stack: local variables, temporary storage, call stack metadata
- But what if you needed a place to store long-lived dynamic memory?

Use mmap()!?!?

- `mmap(0, num_pages*0x1000, ...)`
- + Allows dynamic allocation/deallocation according to changing program needs.
- + Allocated memory survives across functions.
- Inflexible allocation size (must be multiples of 4096 bytes!)
- Very slow! (requires kernel involvement)

Smarter solution

- What if we...

... wrote a library...

... that `mmap()`ed a bunch of memory...

... and handed out small chunks of it...

... on demand!

- The library could be used like:

```
char *firstname = allocate_memory(128);  
char *lastname = allocate_memory(256);  
scanf("%s %s", firstname, lastname);  
printf("Hello %s %s!", firstname, lastname);  
free_memory(firstname);  
free_memory(lastname);
```

firstname	lastname	<i>mmap()ed but unassigned</i>
-----------	----------	--------------------------------

- Dynamic Allocator!

- ptmalloc (Posix Thread Aware fork of `dlmalloc()`) in Linux

What does the heap do?

- The heap, as implemented by ptmalloc/glibc (and analogues), provides:
 - `malloc()` - allocate some memory
 - `free()` - free a prior allocated chunk
- And some auxiliary functions:
 - `realloc()` - change the size of an allocation
 - `calloc()` - allocate and zero-out memory
- These functions are used, extensively, by practically every single non-trivial piece of software.

How does the heap work?

- `ptmalloc` actually does not use `mmap`!
- The Data Segment:
 - with ASLR, placed randomly into memory near-ish the PIE (Position Independent Executable) base
 - starts out with a size of 0 Here, “break” means break point, not **break**; in your switch statement
 - managed by the `brk` and `sbrk` (segment break) system calls:
 - `sbrk(NULL)` returns the end of the data segment
 - `sbrk(delta)` expands the end of the data segment by `delta` bytes
 - `brk(addr)` expands the end of the data segment to `addr`
- Under the hood, this is managed just like `mmap()`.
- `ptmalloc` slices off bits of the data segment for small allocations, and uses `mmap()` for large allocations.

What can go wrong in the heap?

- The heap is:
 1. used by imperfect human programmers
 - humans forget to free memory
 - humans forget all the spots where they store pointers to data
 - humans forget what they've freed
 2. a library that strives for performance
 - allocation and deallocation needs to be fast, or programs will slow down
 - optimizations often leave security as an afterthought
- Bugs caused by #1 become security issues due to #2 if not caught!

Dangers of the heap

- Forgetting to free memory.
 - leads to resource exhaustion
- Forgetting that we have freed memory.
 - using free memory
 - freeing free memory
- Corrupting metadata used by the allocator to keep track of heap state.
 - conceptually similar to corruption internal function state on the stack

Memory Leaks

- Problem: Allocated memory must be explicitly freed.

```
int foo()  
{  
    char *leak = malloc(1024);  
    // use leak in safe ways  
    return 1;  
}
```

In programming languages with garbage collection
(Like Python and Java), memory is implicitly
freed.

- What happens with the memory pointed to by leak?
- Why is this a security issue?

Use After Free (UAF)

- Pointers to an allocation remain valid after `free()`ing the allocation, and might be used afterwards! Why is this bad?

```
int main() {
    char *user_input = malloc(8);

    printf("Name? ");
    scanf("%7s", user_input);
    printf("Hello %s!\n", user_input);
    free(user_input);

    long *authenticated = malloc(8);
    *authenticated = 0;

    printf("Password? ");
    scanf("%7s", user_input);

    if (getuid() == 0 || strcmp(user_input, "hunter2") == 0) *authenticated = 1;
    if (*authenticated) sendfile(0, open("/flag", 0), 0, 128);
}
```

Use After Free (UAF)

- Pointers to an allocation remain valid after `free()`ing the allocation, and might be used afterwards! Why is this bad?

```
int main() {  
    char *user_input = malloc(8);
```

The program frees `user_input`, but then uses it again.

```
    printf("Name? ");  
    scanf("%7s", user_input);  
    printf("Hello %s!\n", user_input);  
    free(user_input);
```

`Authenticated = malloc(8)` may return the same memory that `user_input` had, since it was just freed.

Now:

```
    long *authenticated = malloc(8);  
    *authenticated = 0;
```

- `authenticated` and `user_input` point to the same memory.
- When the attacker inputs a password, `scanf("%7s", user_input)` writes to the same memory that holds the authenticated flag.
- This means the attacker can overwrite the authentication variable by anything non-zero.

```
    printf("Password? ");  
    scanf("%7s", user_input);
```

```
    if (getuid() == 0 || strcmp(user_input, "hunter2") == 0) *authenticated = 1;  
    if (*authenticated) sendfile(0, open("/flag", 0), 0, 128);
```

```
}
```


Memory Disclosure

- **Simple case:** UAF, but with a printf() instead of a scanf().
- **Complex case:** Some heap implementations (including dlmalloc and ptmalloc) reuse free()d chunks to store metadata.

```
int main() {
    char *password = malloc(8);
    char *name = malloc(8);

    printf("Password? ");
    scanf("%7s", password);
    assert(strcmp(password, "hunter2") == 0);
    free(password);

    printf("Name? ");
    scanf("%7s", name);
    printf("Hello %s!\n", name);
    free(name);

    printf("Goodbye, %s!\n", name);
}
```

Metadata Corruption

- Allocator metadata can be written, not just read.
- First introduced in 2000 by Solar designer:
<https://www.openwall.com/articles/JPEG-COM-Marker-Vulnerability>
- Soon formalized in hacker literature in 2001:
 - Vudo malloc tricks, by MaXX: <http://phrack.org/issues/57/8.html>
 - Once upon a free(), by anonymous: <http://phrack.org/issues/57/9.html>
- Now a whole genre in the hacking scene!

Overlapping Allocations

- Typically, heap metadata corruption is used to confuse the allocator into allocating overlapping memory. As we saw with UAF, this can be extremely dangerous.
- Two pointers are pointing to the same memory
 - one is treated by the program in a security-critical manner
 - one can be written to or read by an attacker
- "Security-critical manner?"
 - authentication variables
 - function pointers (control flow hijack)
 - program metadata such as length (inducing memory errors)
 - sensitive data (such as the flag)

tcache

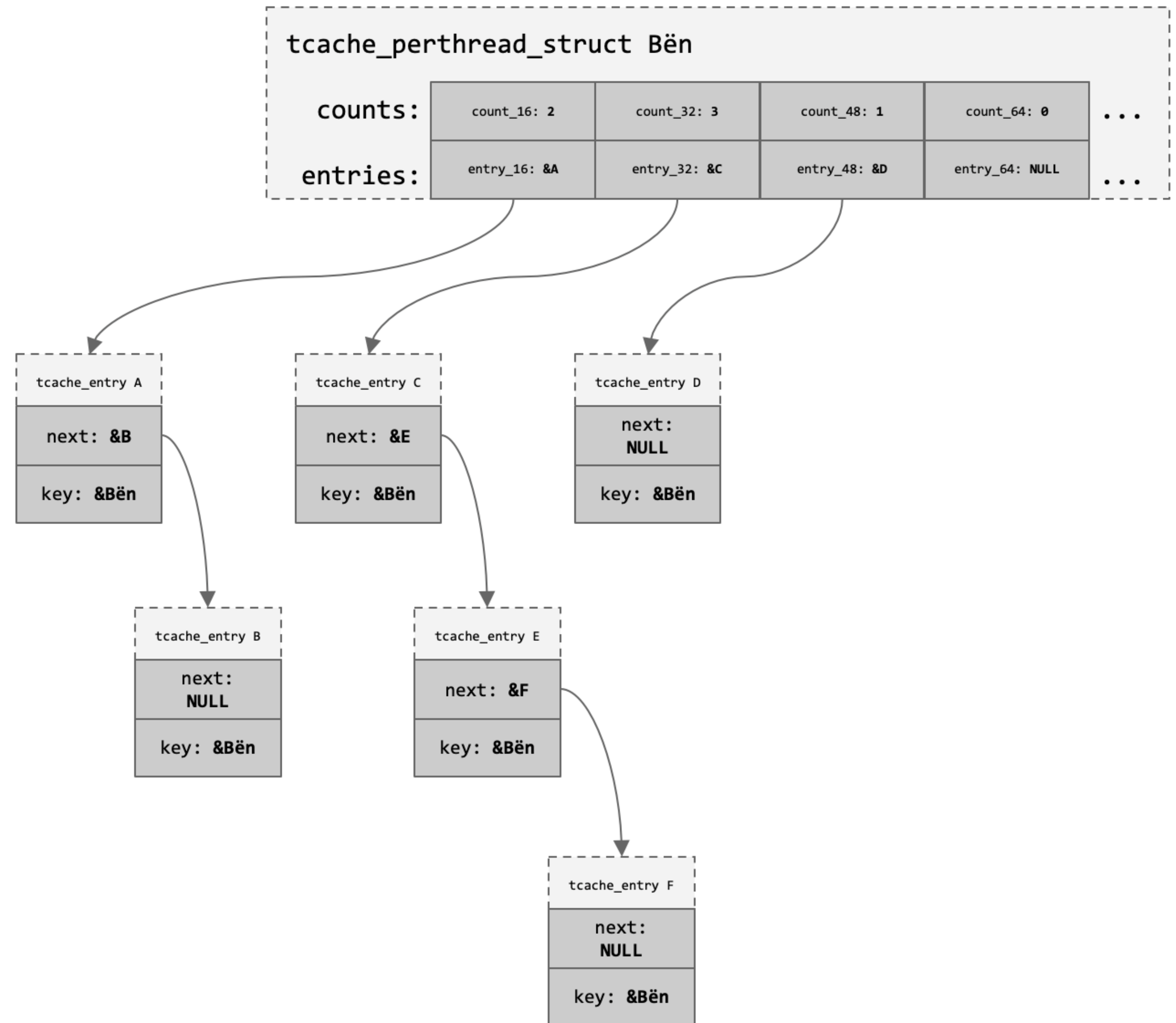
- "Thread Local Caching" in ptmalloc, to speed up repeated (small) allocations in a single thread.
- Tcache is a **temporary holding area** for recently freed memory chunks, organized by size.
- It's designed so that the next call to malloc() of the same size can quickly reuse one of those chunks, without involving slower, global heap operations.
- Implemented as a **singly-linked** list, with each thread having a list header for different-sized allocations.

```
typedef struct tcache_perthread_struct
{
    char counts[TCACHE_MAX_BINS];
    tcache_entry *entries[TCACHE_MAX_BINS];
} tcache_perthread_struct;
```

tcache

```
typedef struct tcache_perthread_struct  
{  
    char counts[TCACHE_MAX_BINS];  
    tcache_entry *entries[TCACHE_MAX_BINS];  
} tcache_perthread_struct;
```

```
typedef struct tcache_entry  
{  
    struct tcache_entry *next;  
    struct tcache_perthread_struct *key;  
} tcache_entry;
```



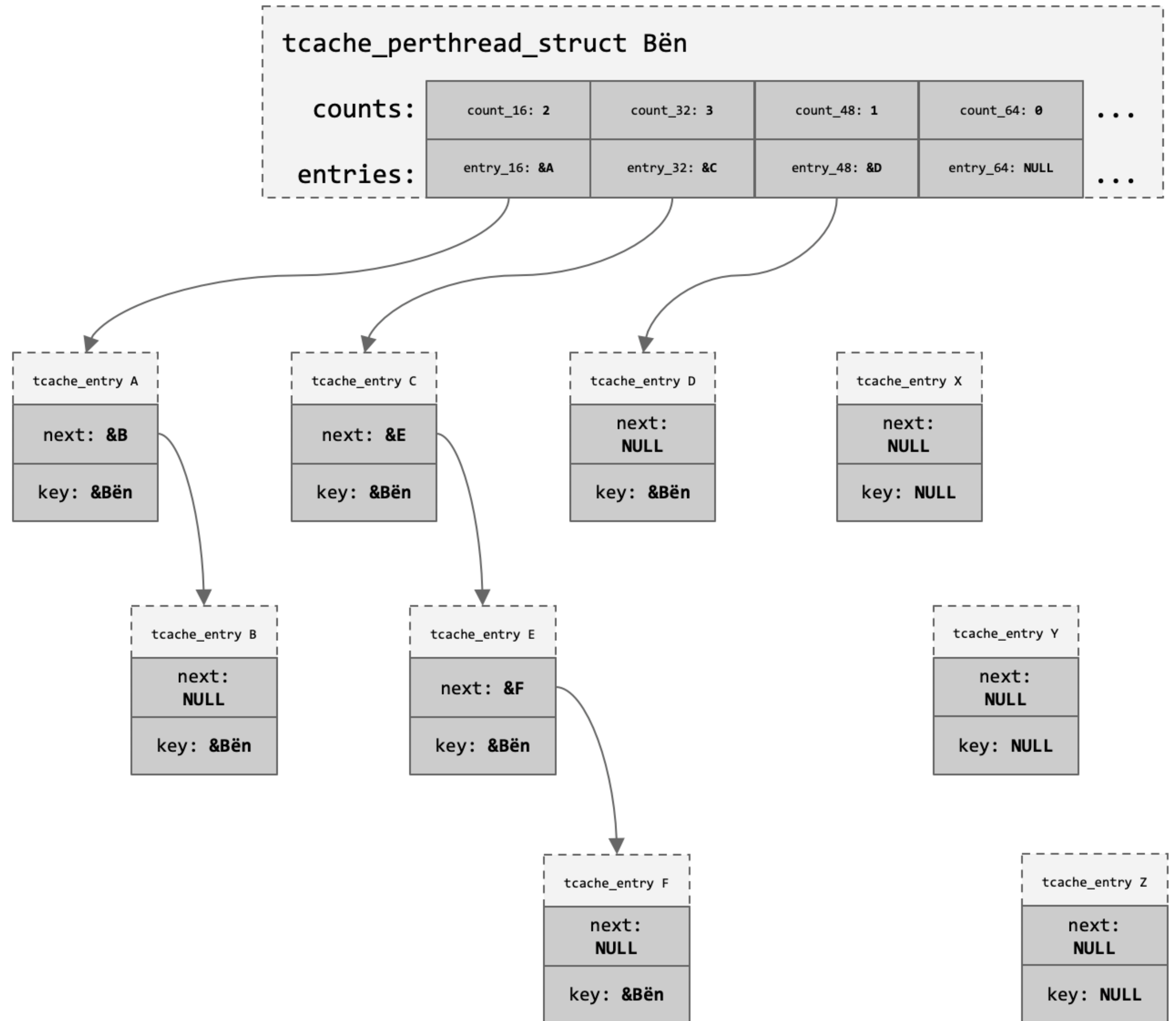
tcache

```
a = malloc(16);  
b = malloc(16);  
c = malloc(32);  
d = malloc(48);  
e = malloc(32);  
f = malloc(32);
```

```
// allocations that are not freed  
// don't show up in the tcache!
```

```
x = malloc(64);  
y = malloc(64);  
z = malloc(64);
```

```
// later freed allocations show up  
// earlier in the tcache list order  
free(b);  
free(a);  
free(f);  
free(e);  
free(c);  
free(d);
```



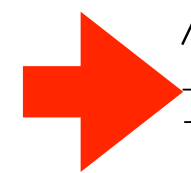
tcache

```
a = malloc(16);  
b = malloc(16);  
c = malloc(32);  
d = malloc(48);  
e = malloc(32);  
f = malloc(32);
```

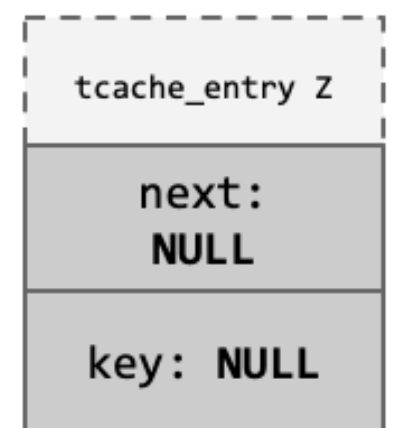
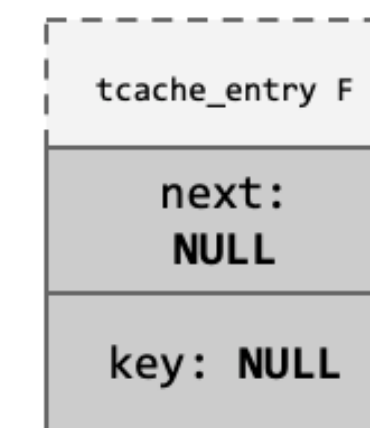
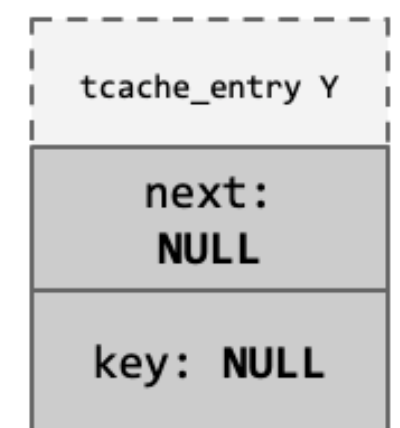
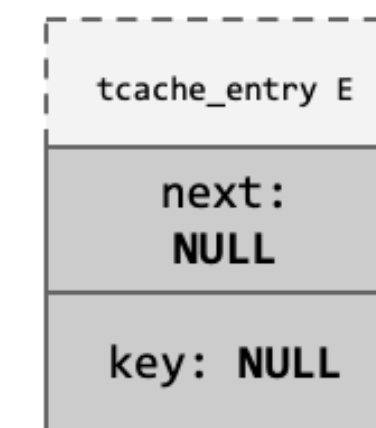
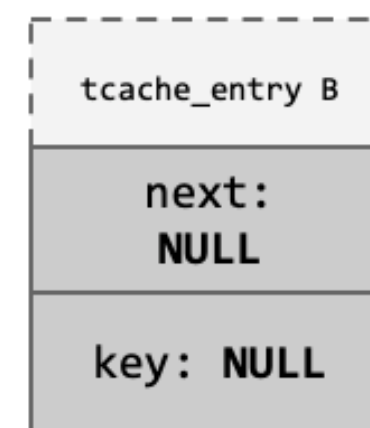
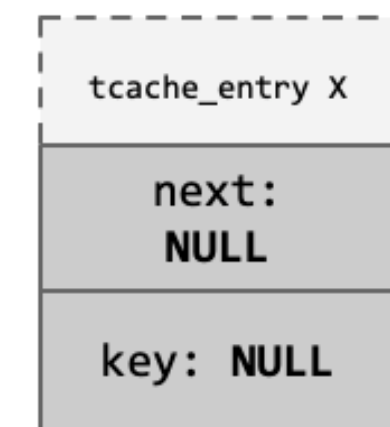
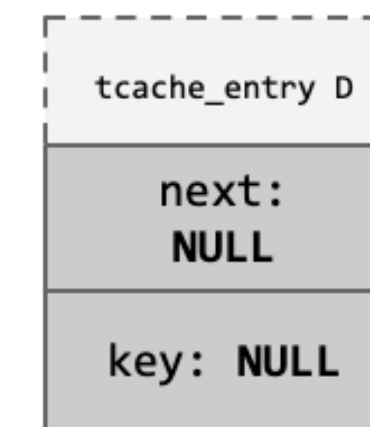
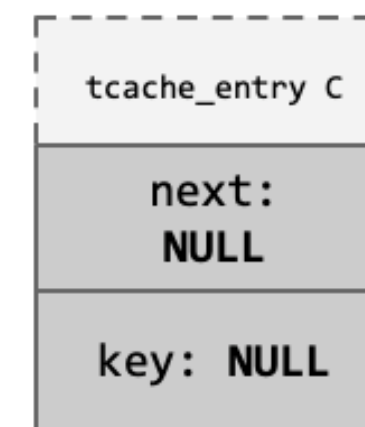
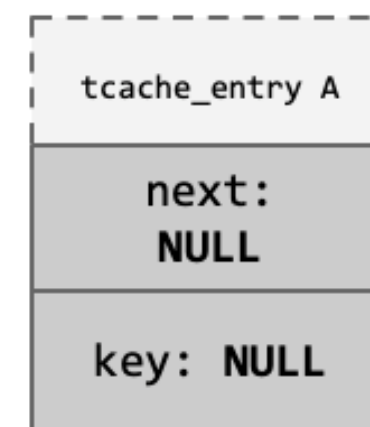
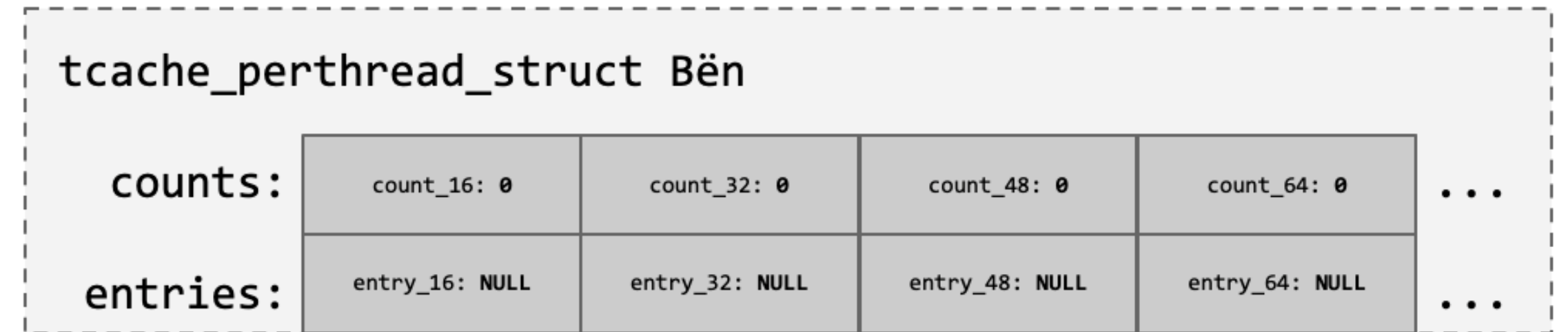
```
// allocations that are not freed  
// don't show up in the tcache!
```

```
x = malloc(64);  
y = malloc(64);  
z = malloc(64);
```

```
// later freed allocations show up  
// earlier in the tcache list order
```



```
free(b);  
free(a);  
free(f);  
free(e);  
free(c);  
free(d);
```



tcache

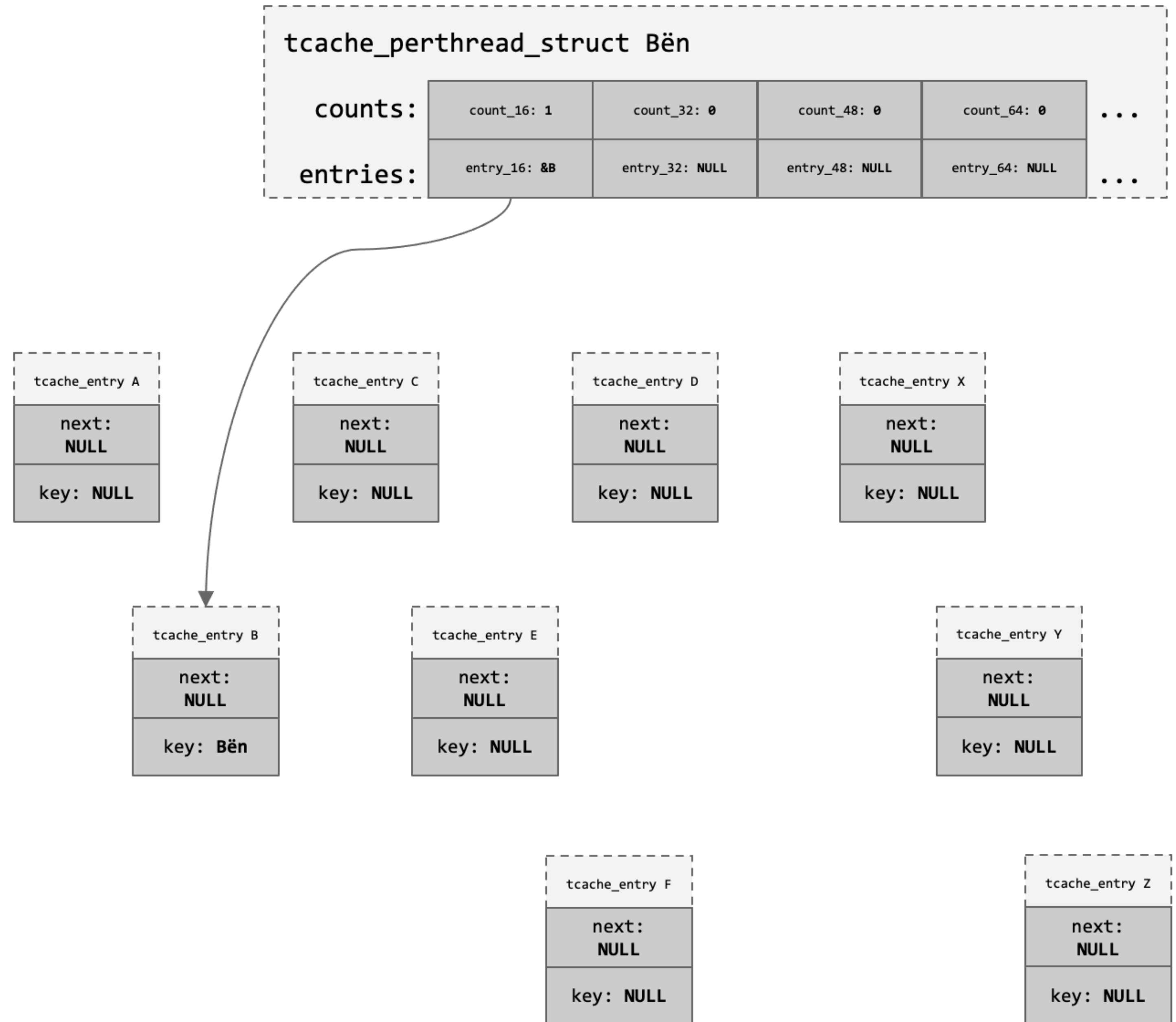
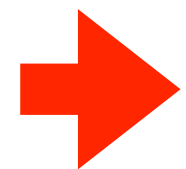
```
a = malloc(16);  
b = malloc(16);  
c = malloc(32);  
d = malloc(48);  
e = malloc(32);  
f = malloc(32);
```

```
// allocations that are not freed  
// don't show up in the tcache!
```

```
x = malloc(64);  
y = malloc(64);  
z = malloc(64);
```

```
// later freed allocations show up  
// earlier in the tcache list order
```

```
free(b);  
free(a);  
free(f);  
free(e);  
free(c);  
free(d);
```



tcache

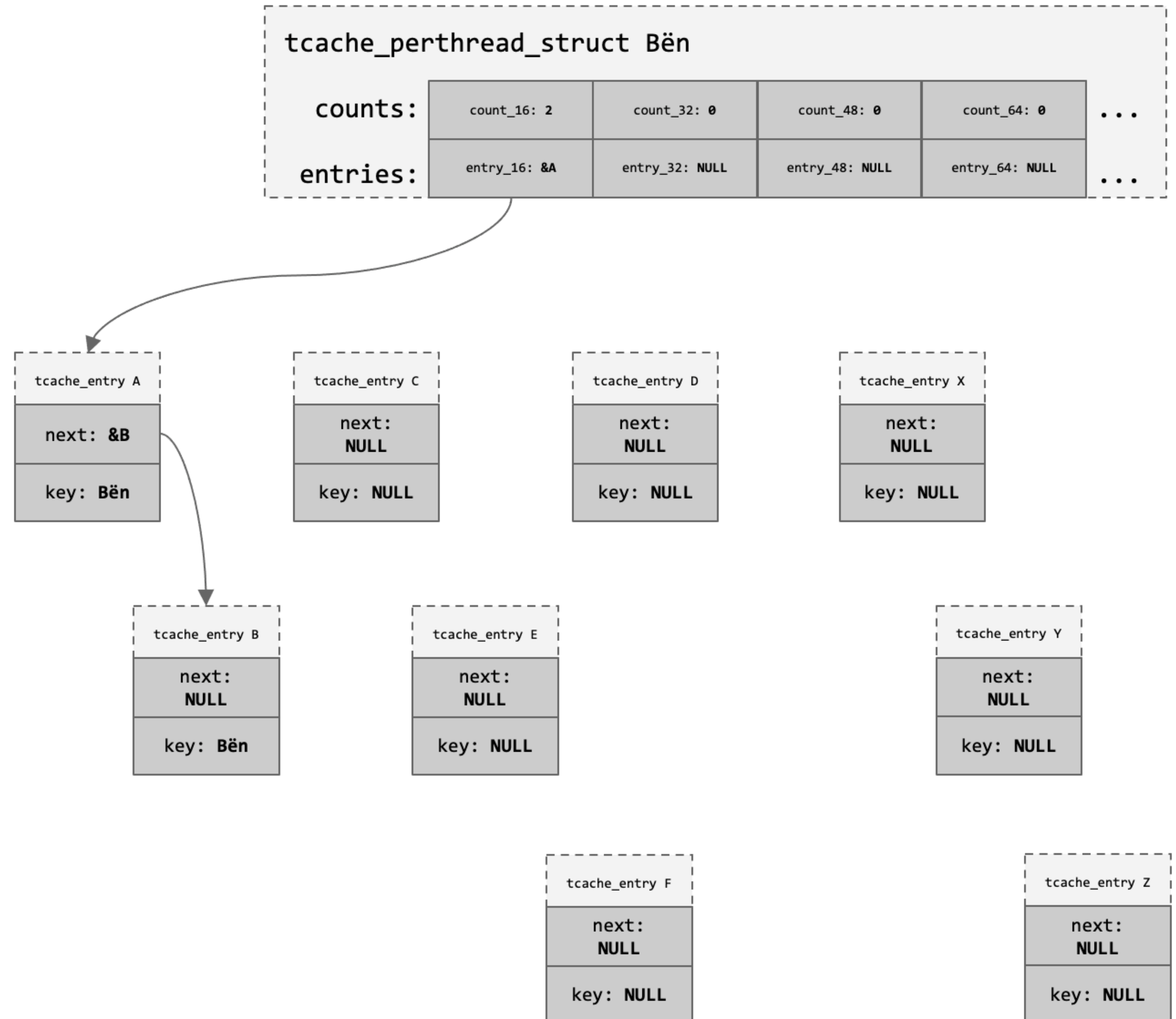
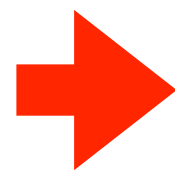
```
a = malloc(16);  
b = malloc(16);  
c = malloc(32);  
d = malloc(48);  
e = malloc(32);  
f = malloc(32);
```

```
// allocations that are not freed  
// don't show up in the tcache!
```

```
x = malloc(64);  
y = malloc(64);  
z = malloc(64);
```

```
// later freed allocations show up  
// earlier in the tcache list order
```

```
free(b);  
free(a);  
free(f);  
free(e);  
free(c);  
free(d);
```



tcache

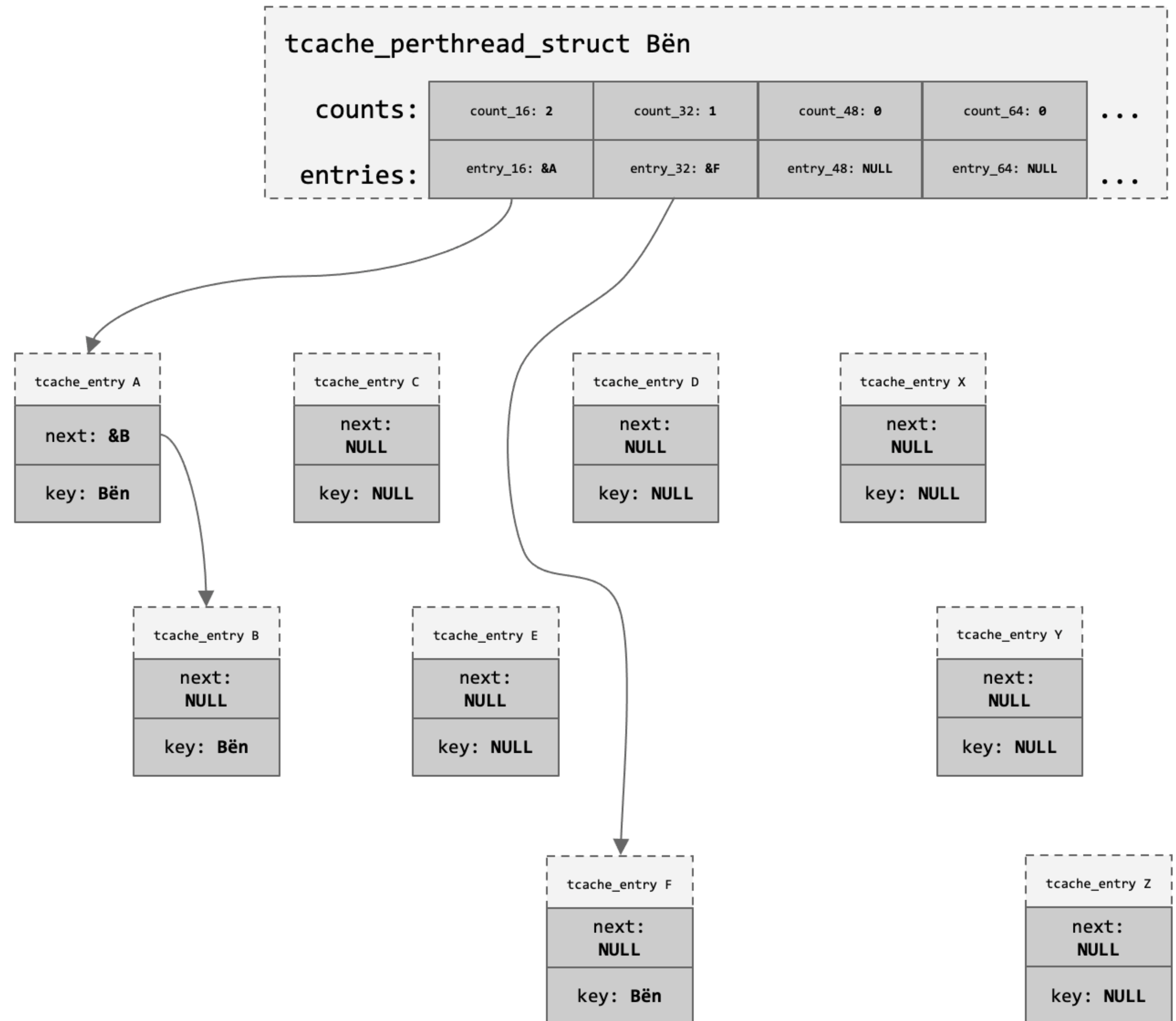
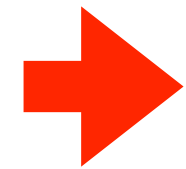
```
a = malloc(16);  
b = malloc(16);  
c = malloc(32);  
d = malloc(48);  
e = malloc(32);  
f = malloc(32);
```

```
// allocations that are not freed  
// don't show up in the tcache!
```

```
x = malloc(64);  
y = malloc(64);  
z = malloc(64);
```

```
// later freed allocations show up  
// earlier in the tcache list order
```

```
free(b);  
free(a);  
free(f);  
free(e);  
free(c);  
free(d);
```



tcache

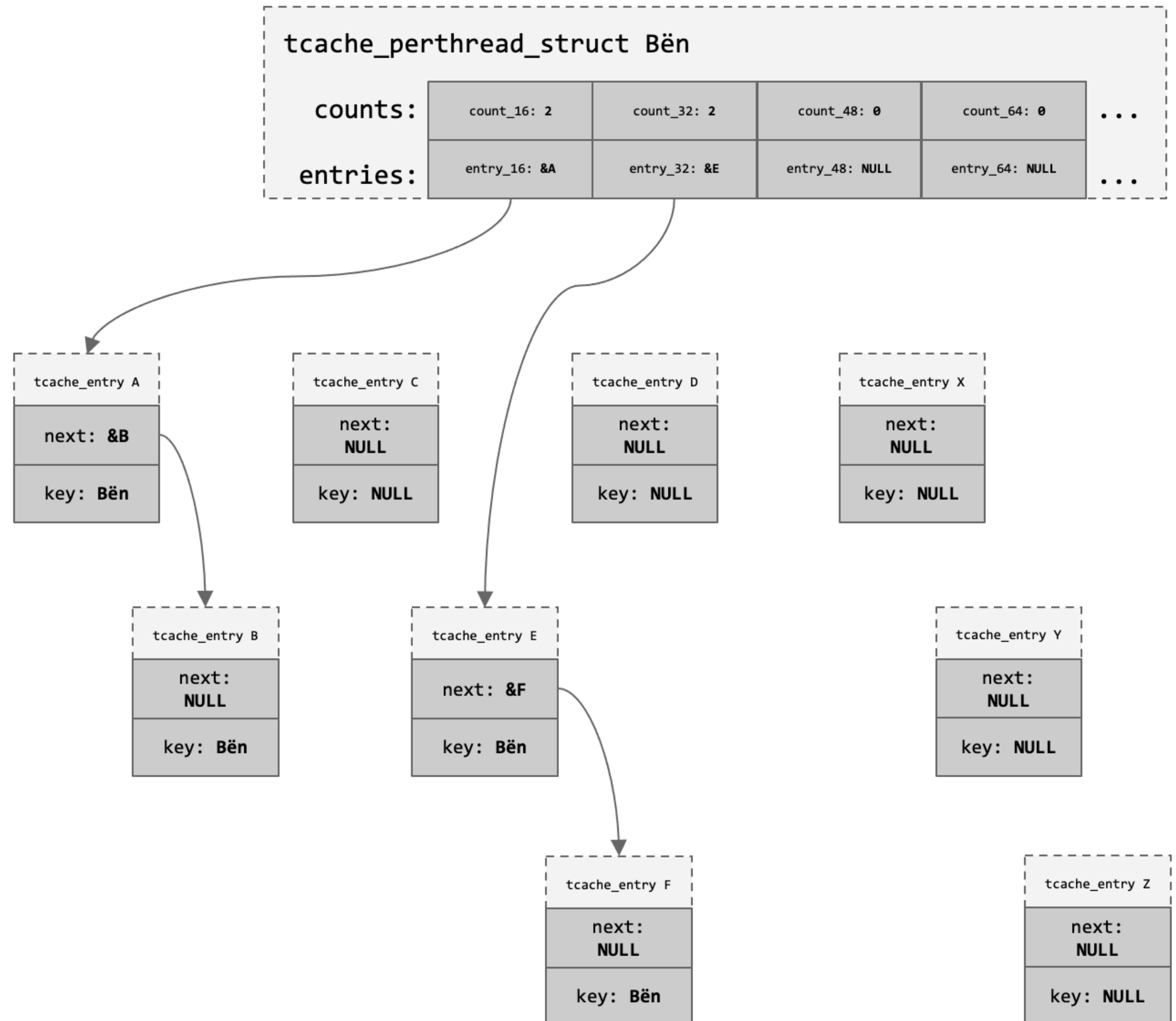
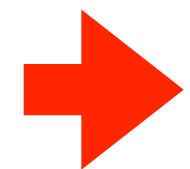
```
a = malloc(16);  
b = malloc(16);  
c = malloc(32);  
d = malloc(48);  
e = malloc(32);  
f = malloc(32);
```

```
// allocations that are not freed  
// don't show up in the tcache!
```

```
x = malloc(64);  
y = malloc(64);  
z = malloc(64);
```

```
// later freed allocations show up  
// earlier in the tcache list order
```

```
free(b);  
free(a);  
free(f);  
free(e);  
free(c);  
free(d);
```



tcache

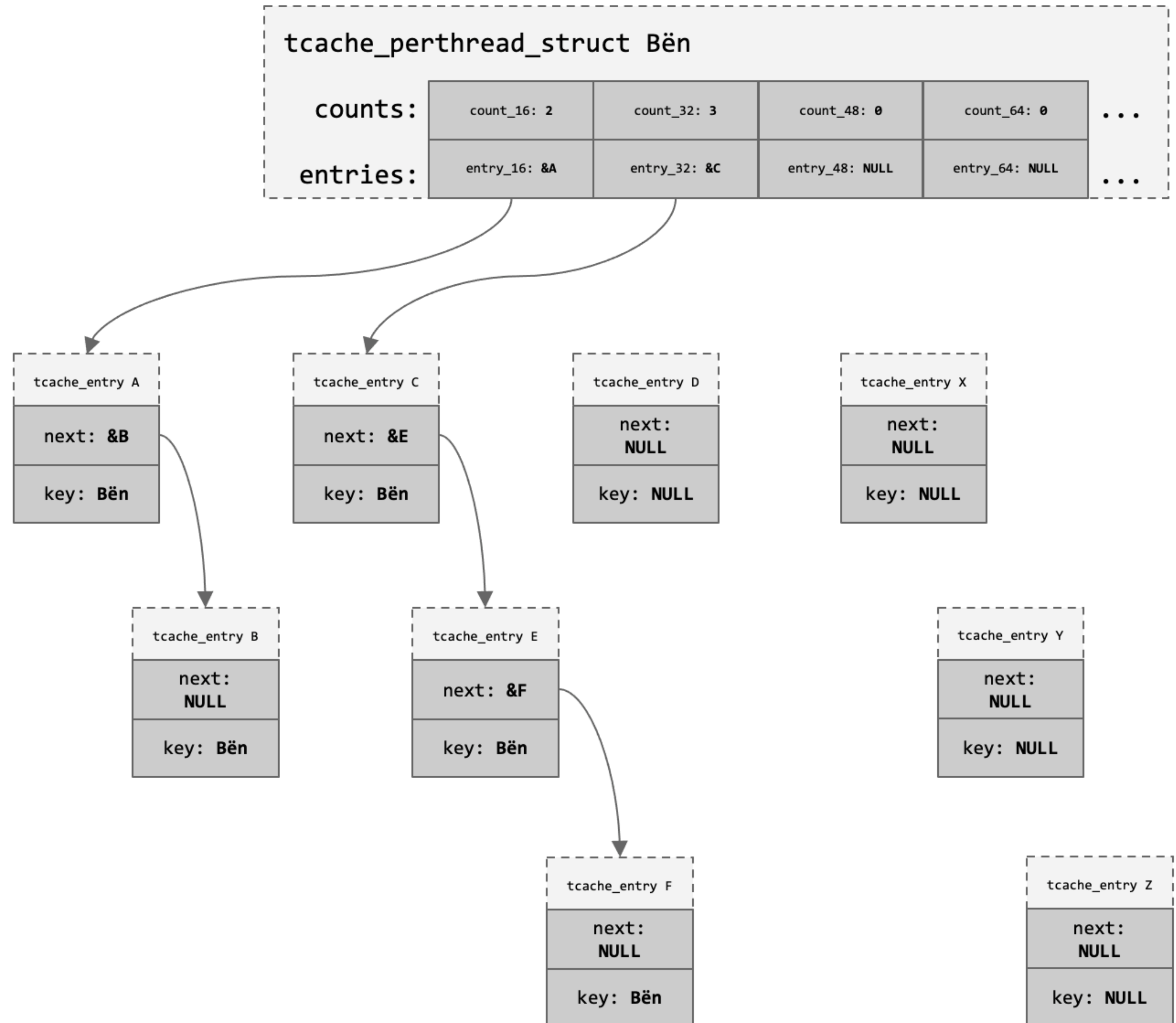
```
a = malloc(16);  
b = malloc(16);  
c = malloc(32);  
d = malloc(48);  
e = malloc(32);  
f = malloc(32);
```

```
// allocations that are not freed  
// don't show up in the tcache!
```

```
x = malloc(64);  
y = malloc(64);  
z = malloc(64);
```

```
// later freed allocations show up  
// earlier in the tcache list order
```

```
free(b);  
free(a);  
free(f);  
free(e);  
free(c);  
free(d);
```



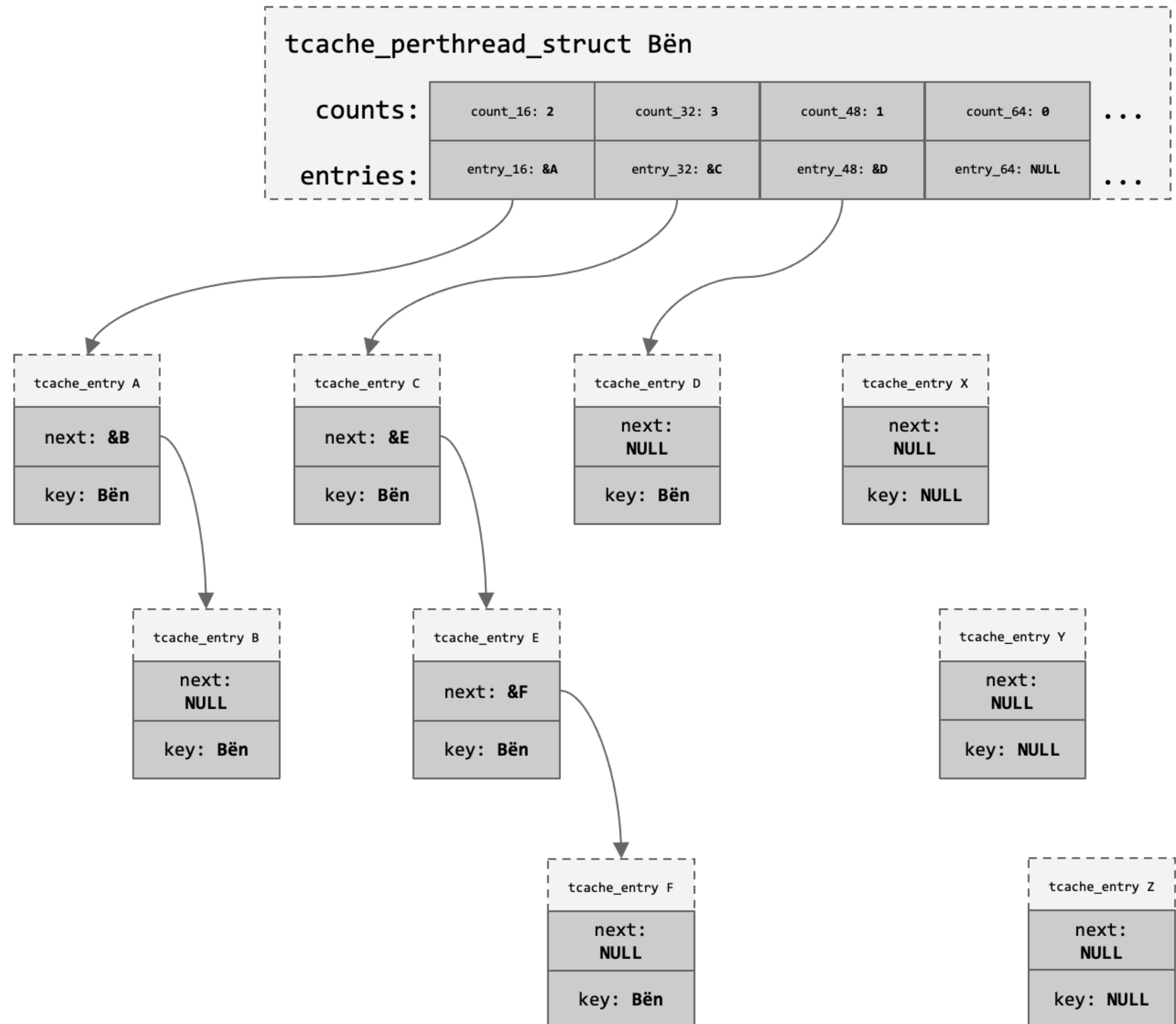
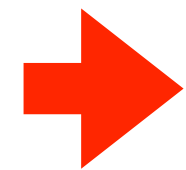
tcache

```
a = malloc(16);  
b = malloc(16);  
c = malloc(32);  
d = malloc(48);  
e = malloc(32);  
f = malloc(32);
```

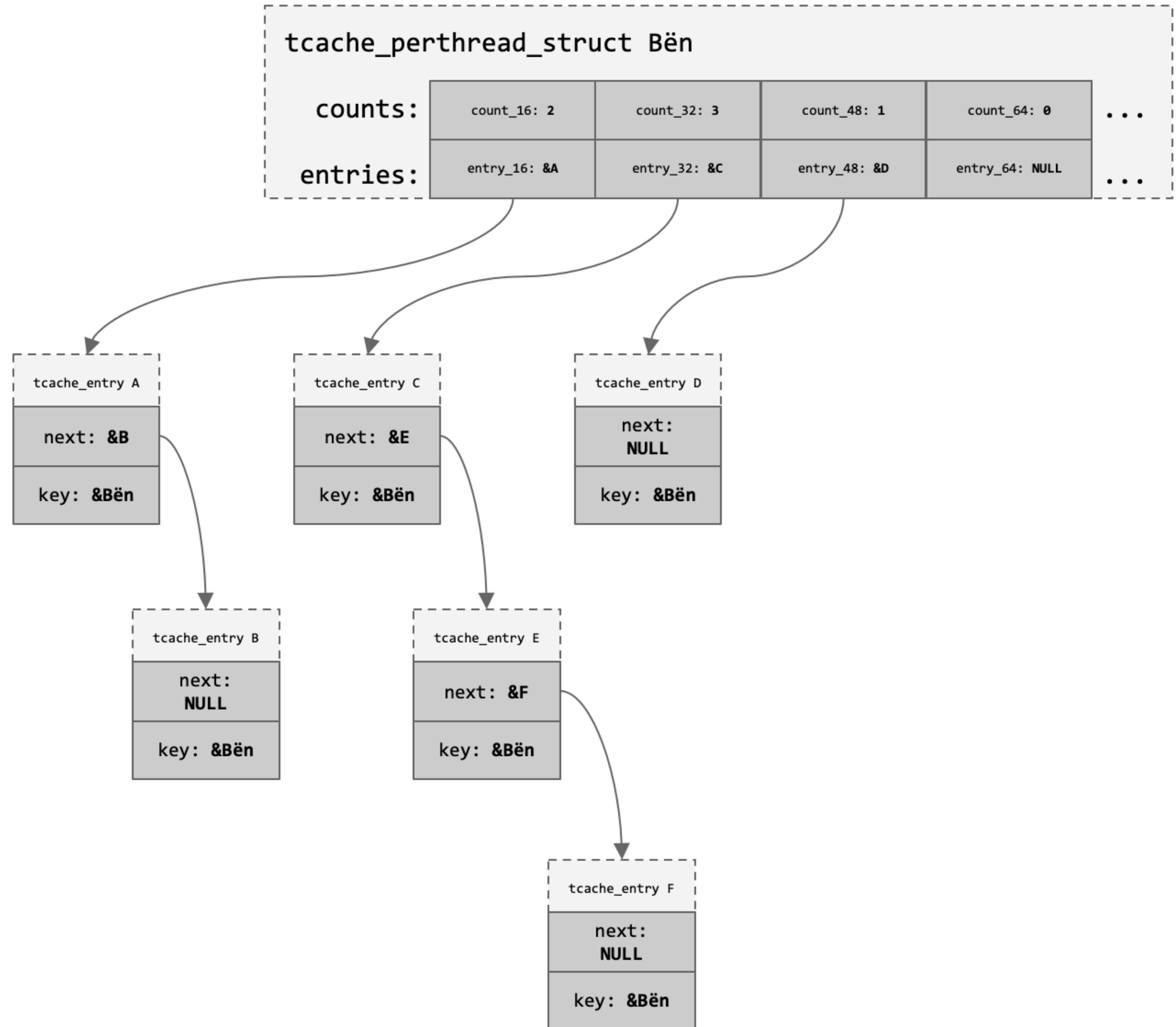
```
// allocations that are not freed  
// don't show up in the tcache!
```

```
x = malloc(64);  
y = malloc(64);  
z = malloc(64);
```

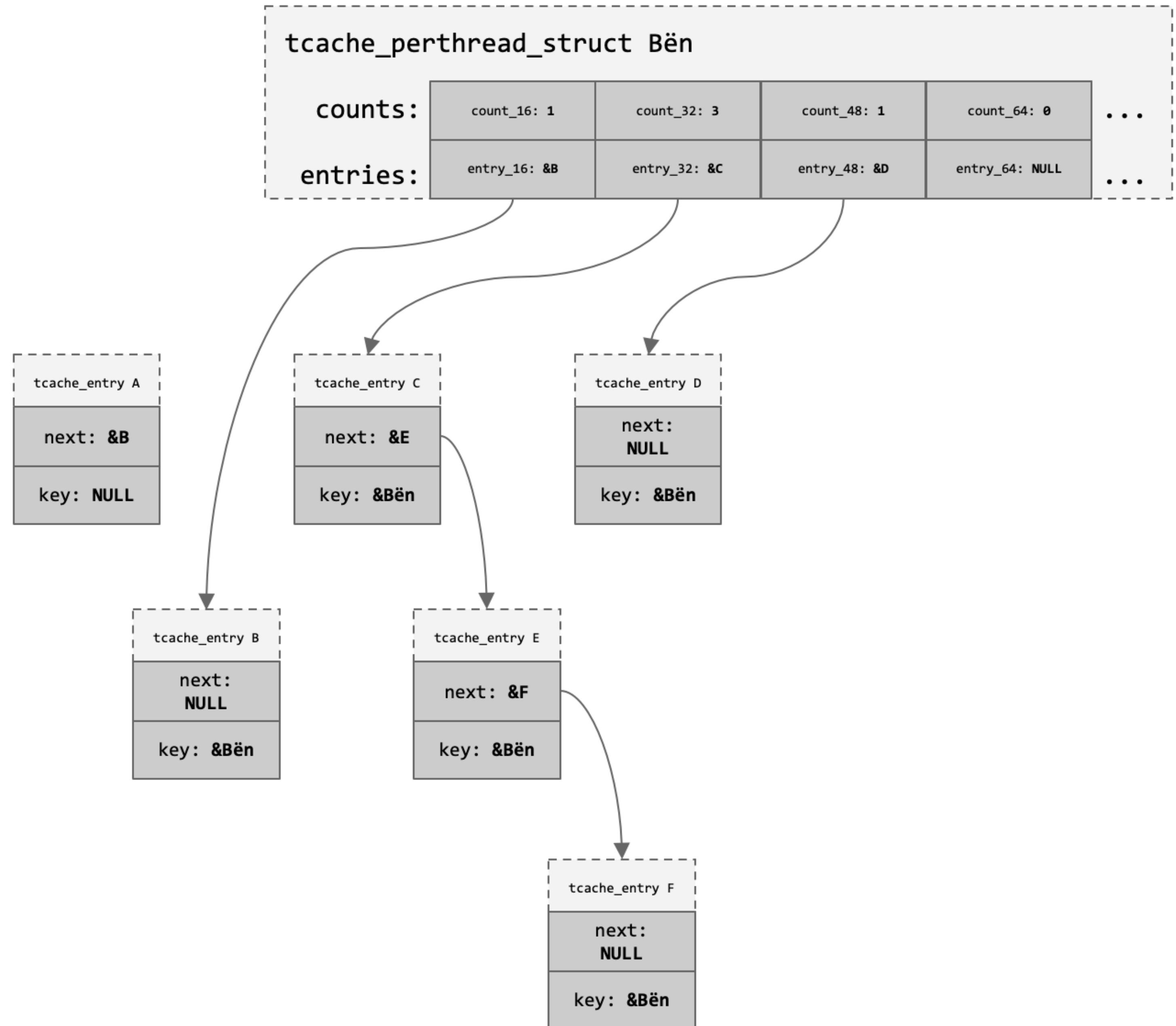
```
// later freed allocations show up  
// earlier in the tcache list order  
free(b);  
free(a);  
free(f);  
free(e);  
free(c);  
free(d);
```



tcache



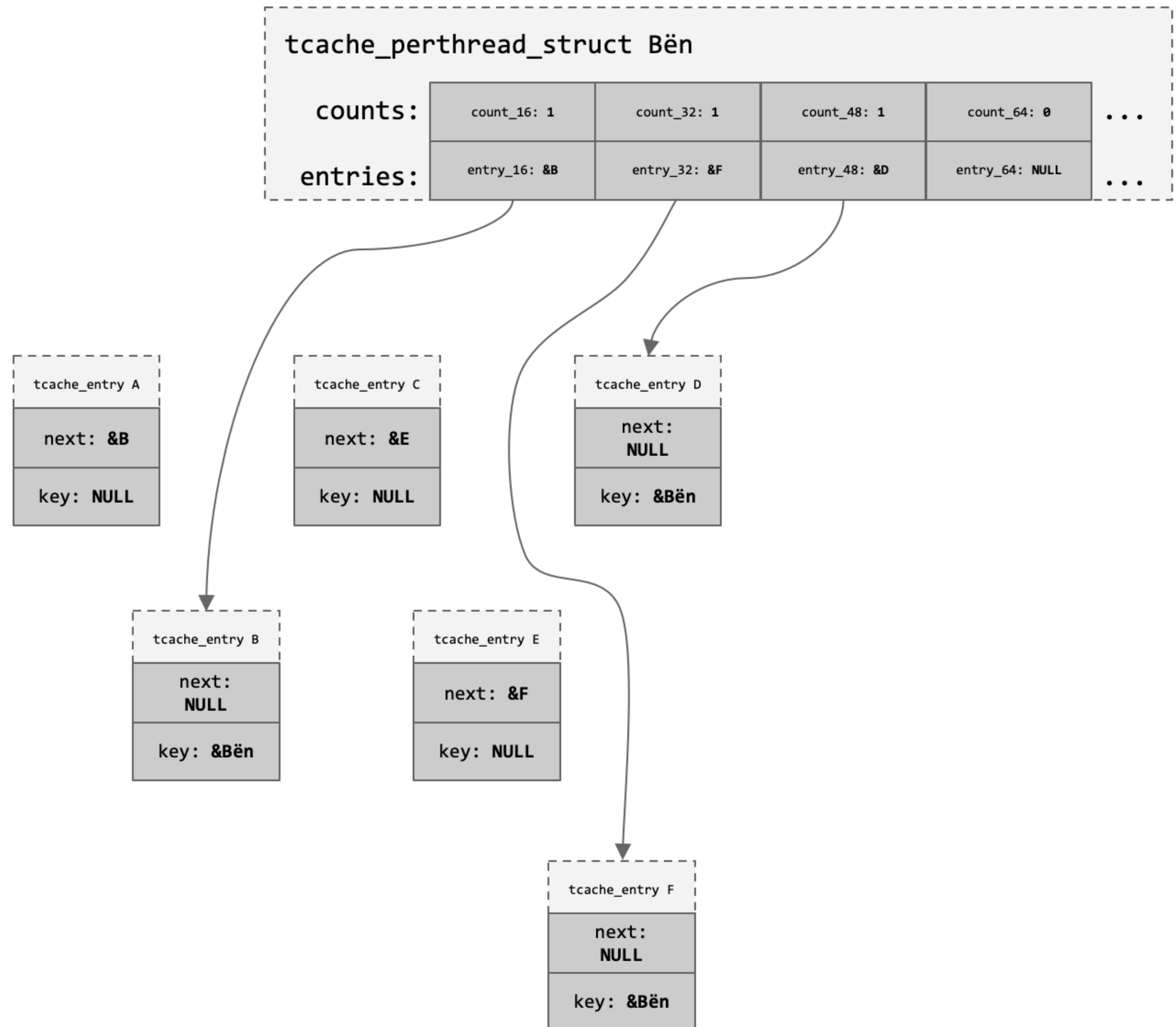
tcache



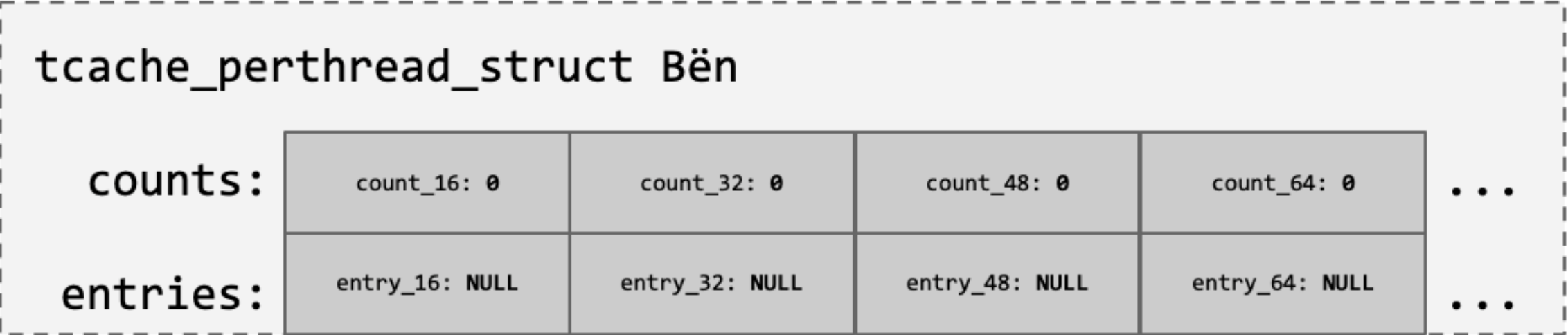
`malloc(16) == a`

tcache

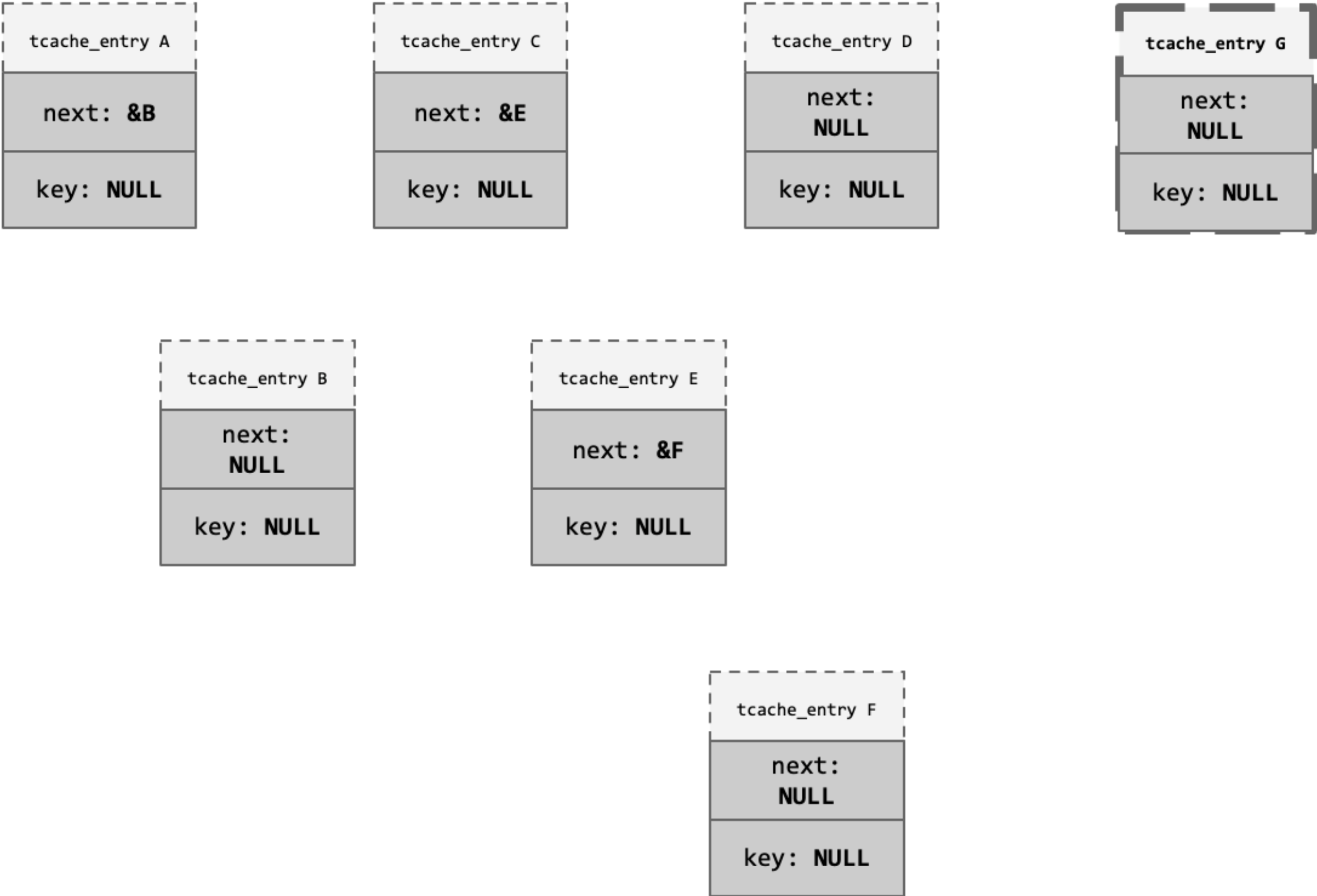
```
malloc(16) == a  
malloc(32) == c  
malloc(32) == e
```



tcache



malloc(16) == a
malloc(32) == c
malloc(32) == e
malloc(48) == d
malloc(16) == b
malloc(32) == f
malloc(64) == g



tcache - freeing

- Each `tcache_entry` is actually the exact allocation that was freed! On `free()`, the following happens:
- **Select** the right "bin" based on the size:
`idx = (freed_allocation_size - 1) / 16;`
- **Push** the freed allocation to the front of the list!
`chunk[0] = tcache.entries[idx];` `// store the current head`
`tcache.entries[idx] = chunk;` `// insert chunk at the front`
`tcache.counts[idx]++;` `// increment count`
- **Record** the `tcache_perthread_struct` associated with the freed allocation (for checking against double-frees)
`chunk[1] = &tcache;`

Double free

- **Problem:** Pointers to an allocation remain valid after `free()`ing the allocation, and are sometimes `free()`d again!
- **Solution:** New versions of glibc/ptmalloc introduced the key check.

tcache - freeing

- Each `tcache_entry` is actually the exact allocation that was freed! On `free()`, the following happens:
- **Select** the right "bin" based on the size:
`idx = (freed_allocation_size - 1) / 16;`
- **Check** to make sure the entry hasn't already been freed (double-free):
`if (chunk[1] == &tcache) //Double free detected`
- **Push** the freed allocation to the front of the list!
`chunk[0] = tcache.entries[idx]; // store the current head`
`tcache.entries[idx] = chunk; // insert chunk at the front`
`tcache.counts[idx]++; // increment count`
- **Record** the `tcache_perthread_struct` associated with the freed allocation (for checking against double-frees)
`chunk[1] = &tcache;`

Double-free

- The check works by looking for the key
- What if we overwrite key after `free()`ing our allocation...

```
int main()
{
    unsigned long long *a;
    a=malloc(128);

    free(a);

    a[1] = 1234;

    free(a);

    printf( "malloc 1: %p/n"  malloc(128));
    printf( "malloc 2: %p/n"  malloc(128));
}
```

- Two `malloc()`s can get the same address

tcache - allocation

- On allocation, the following happens:
- **Select** the bin number based on the requested size:
`idx = (requested_size - 1) / 16;`
- **Check** the appropriate cache for available entries:
`if (tcache.count[idx] > 0)`
- **Reuse** the allocation in the front of the list if available:
`to_return = tcache.entries[idx];` `// head of the bin`
`tcache.entries[idx] = to_return[0];` `// update head to next`
`tcache.count[idx]--;`
`return to_return;`
- Things that are **not** done:
 - clearing all sensitive pointers (only key is cleared).
 - checking if the next (`return[0]`) address makes sense

tcache poisoning

- What happens if we corrupt tcache_entry->next?

```
int main()
{
    char stack_buffer[16];
    unsigned long long *a;
    unsigned long long *b;

    // warm up the tcache
    a = malloc(16);
    b = malloc(16);

    free(b);
    free(a);

    //corrupt the next pointer
    a[0] = &stack_buffer;

    printf("stack buffer: %p/n" &stack_buffer);
    printf("malloc 1: %p/n" malloc(16));
    printf("malloc 2: %p/n" malloc(16));
}
```

//tcache.entries[0] → b → NULL
//tcache.entries[0] → a → b → NULL

//tcache.entries[0] → a → stack buffer

//returns a
//returns stack buffer

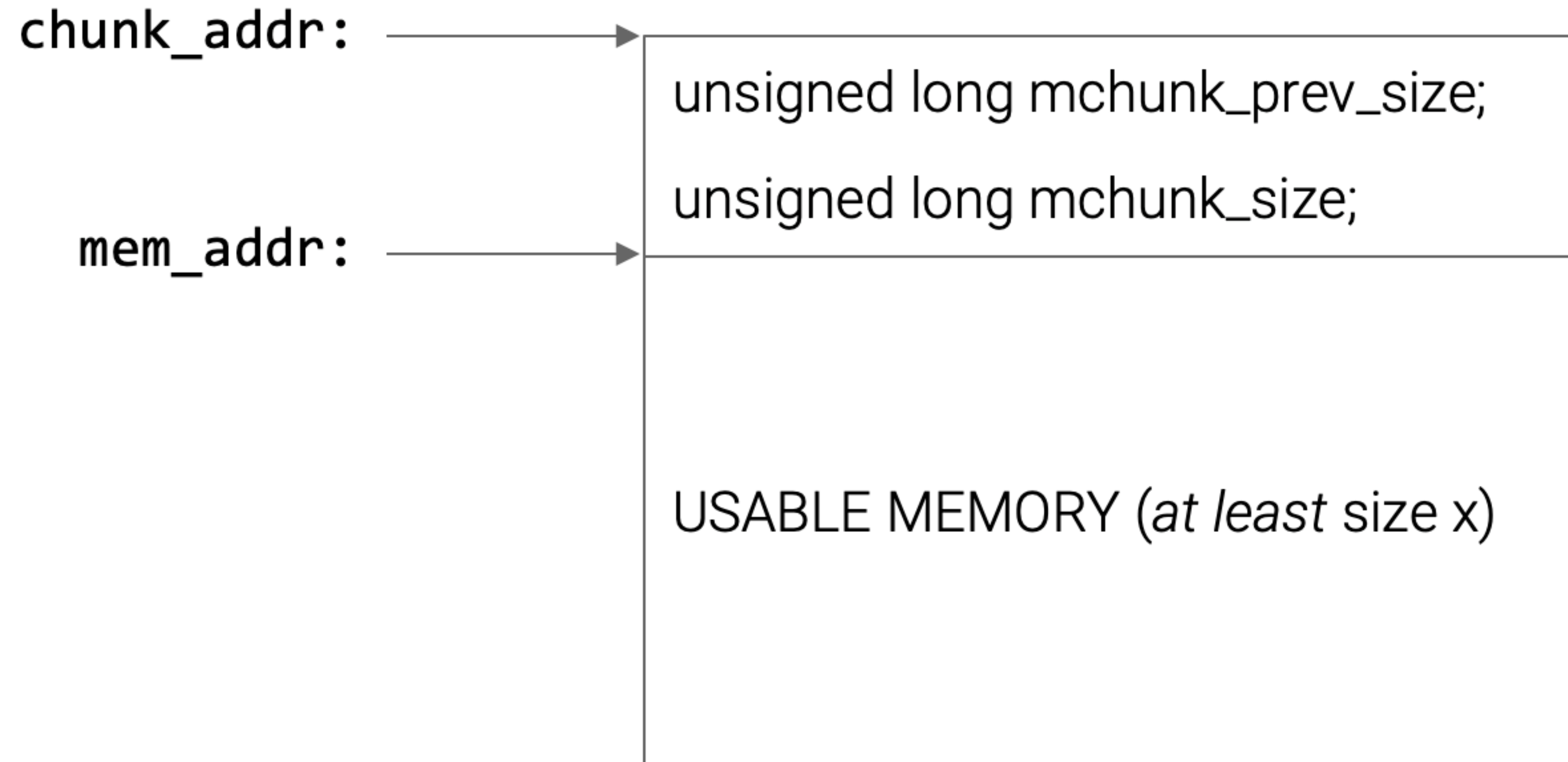
- First malloc() returns a, second one returns what a->next points to.

Heap Metadata and its Corruption

- As we saw with tcache, the ptmalloc uses a bunch of metadata to track its operation. It keeps them in:
 - global metadata (i.e., the tcache structure)
 - per-chunk metadata
- What's a chunk?

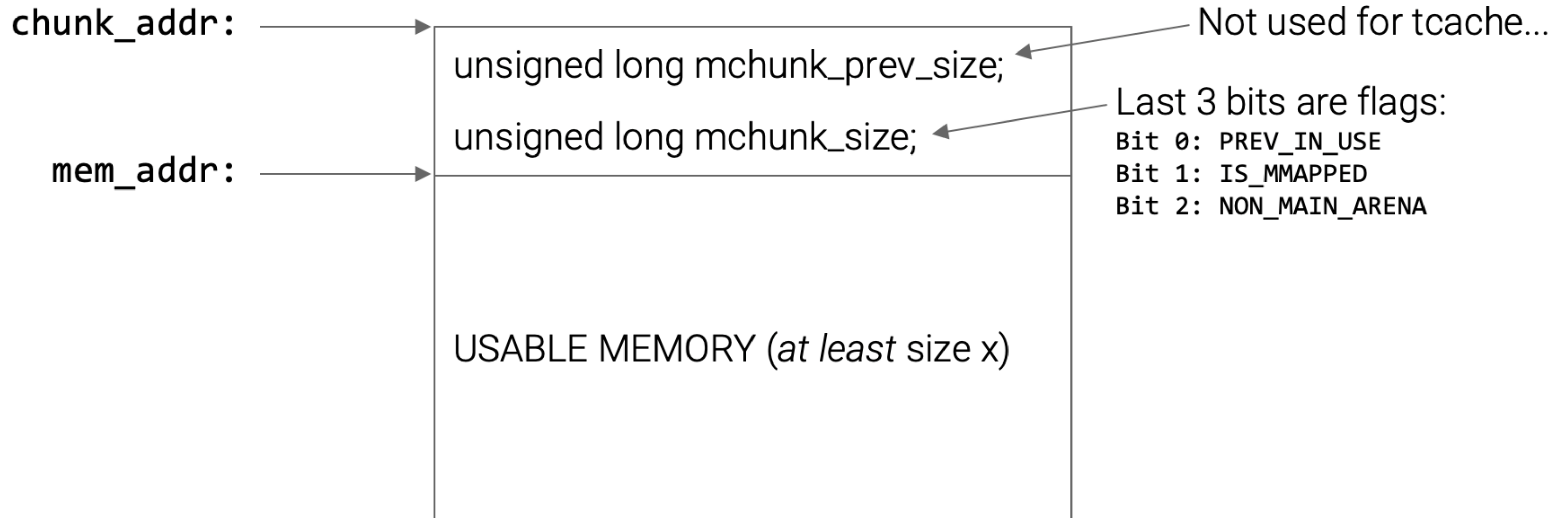
Metadata: Allocated Chunks

- malloc(x) returns mem_addr, but in actuality, ptmalloc tracks chunk_addr:



Metadata: Size

- malloc(n) guarantees at least n usable space, but chunks sizes are multiples of 0x10.



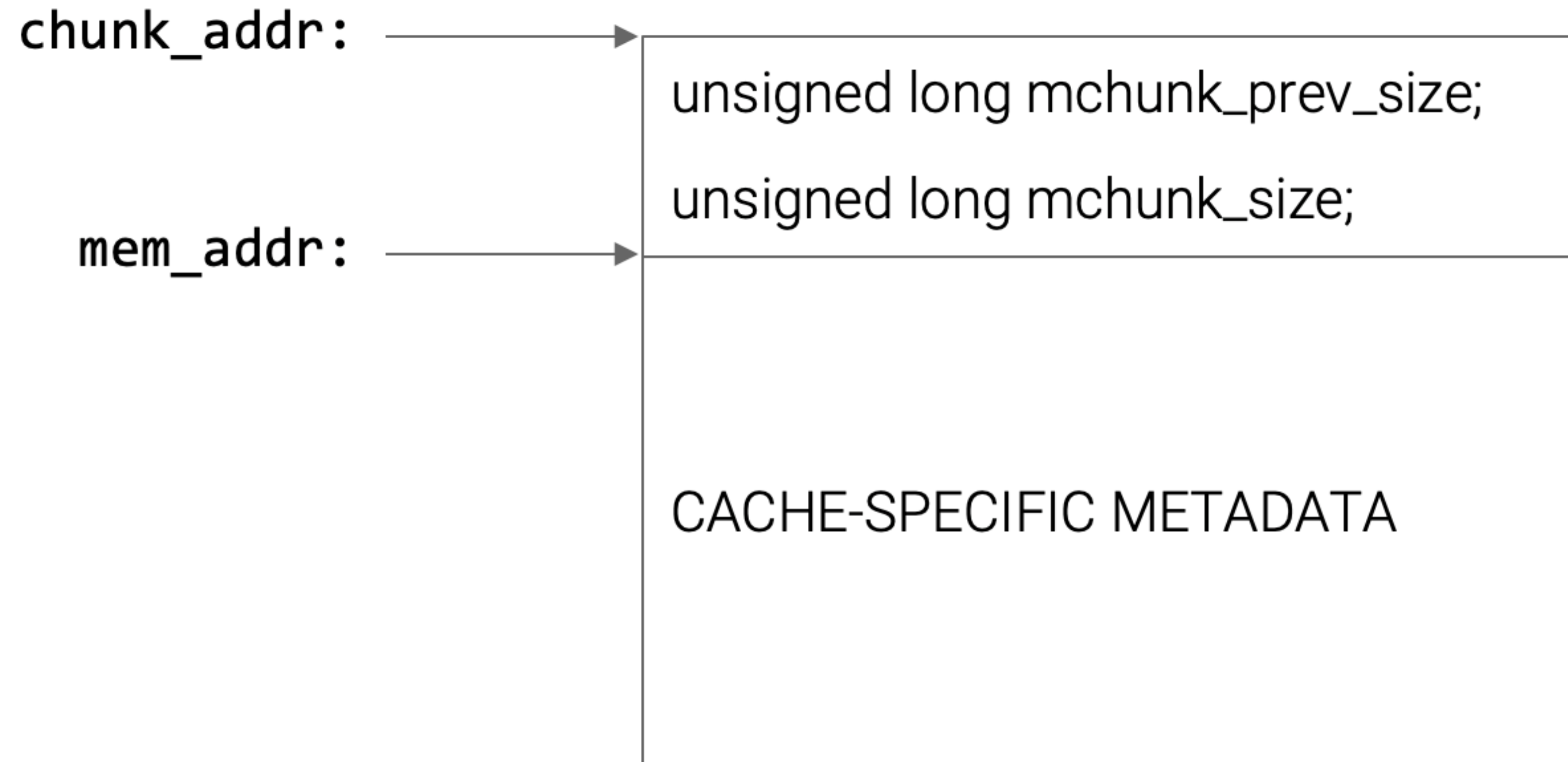
Metadata: Overlapping metadata!

- To save memory, the `prev_size` field of a chunk whose `PREV_INUSE` flag is set (i.e., the previous chunk is not free) **is used by the previous chunk!**

chunk1: unsigned long *a = malloc(0x10)				chunk2: unsigned long *b = malloc(0x10)			
prev_size	size	a[0]	a[1]	prev_size	size	b[0]	b[1]

Metadata: Freed Chunks

- As we saw with tcache, a free()d chunk has additional metadata about the location of other chunks:

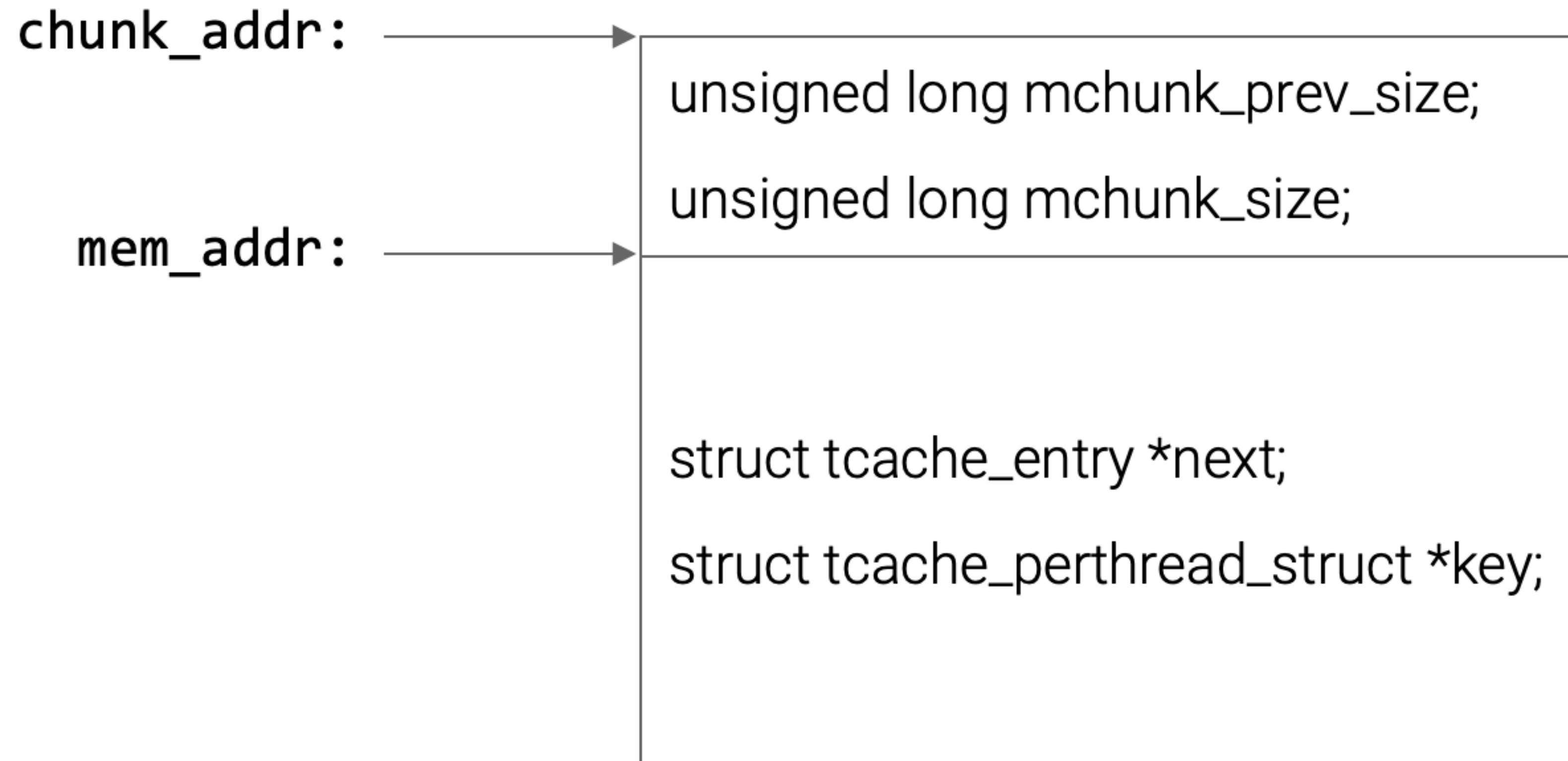


Metadata: Different Caches

- This information is constantly changing (tcache!) and **PTMALLOC IS VERY COMPLEX**. This is an approximation.
- Currently, the ptmalloc caching design is (in order of use):
 1. 64 singly-linked **tcache** bins for allocations of size 16 to 1032
 2. 10 singly-linked **"fast" bins** for allocations of size up to 160 bytes
 3. 1 doubly-linked **"unsorted" bin** to quickly stash free()d chunks that don't fit into tcache or fastbins
 4. 64 doubly-linked **"small" bins** for allocations up to 512 bytes
 5. doubly-linked **"large" bins** (anything over 512 bytes) that contain different-sized chunks

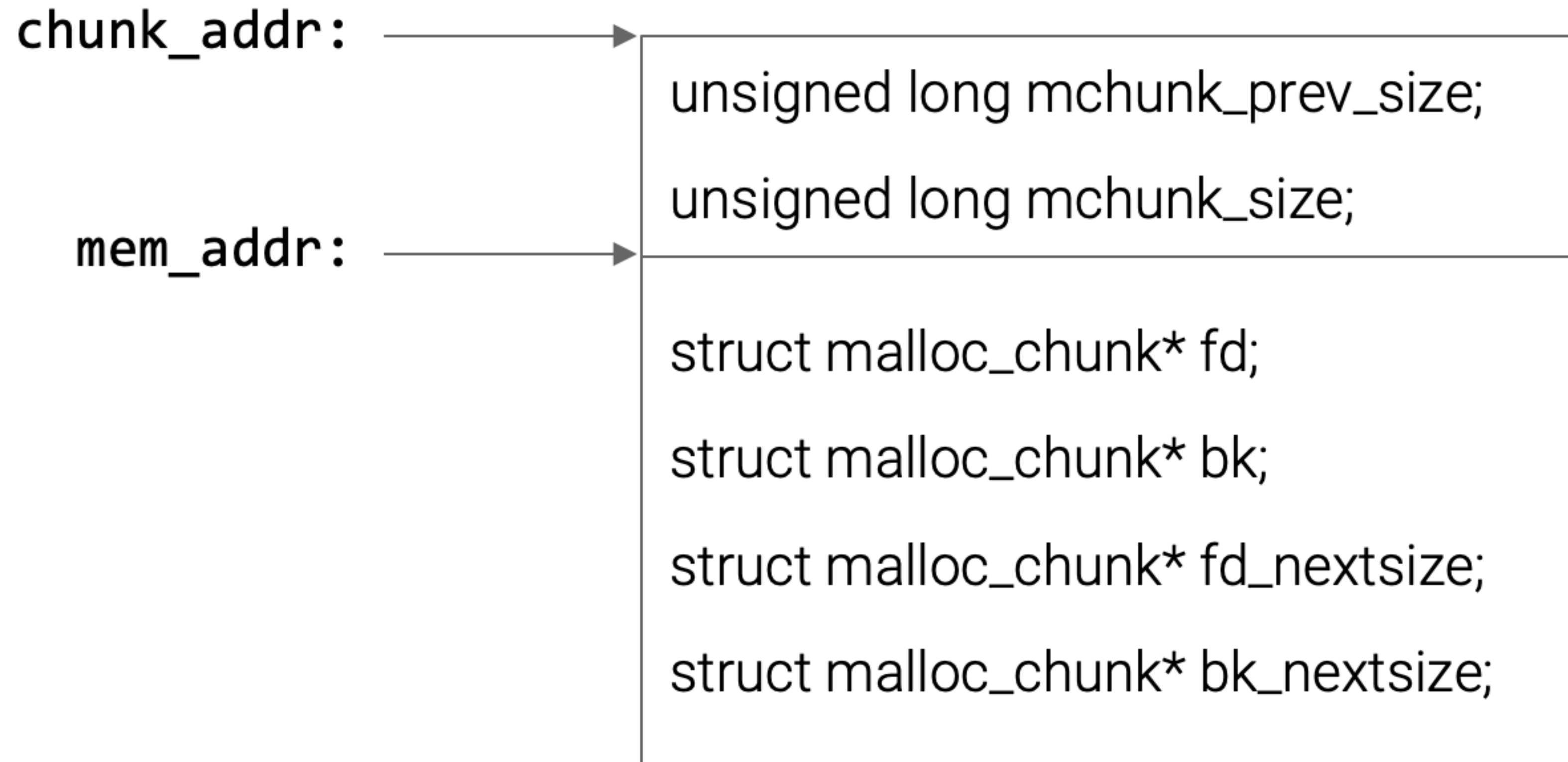
Metadata: tcache Chunks

- Free tcache-cached chunks have a pointer to the allocated space of the next chunk and a pointer to the per-thread struct.



Metadata: largebin Chunks

- When `free()`d, large are:
 1. Consolidated with adjacent free chunks.
 2. Put into an "unsorted" bin regardless of size.
 3. Properly put into a doubly-linked list later, during the next allocation that they fail to "satisfy".



Metadata: The Wilderness

- The heap is a finite-sized allocation that needs to be manually expanded.
- During allocation, malloc() will (simplified view):
 1. Look for a free chunk that will satisfy the allocation, and return it.
 2. Otherwise, check the "available" space left at the end of the heap. If there is enough there, return that and reduce the "available" space.
 3. If there isn't enough "available" space, malloc() will mmap() certain large allocations.
 4. Otherwise, malloc will grow the heap with brk() and go to #2.
- How does malloc() store the available space? In the "Wilderness", a fake chunk at the end of the heap that stores the available space.

Other Allocators?

- Allocators are different! This metadata discussion, and tcache, is very ptmalloc-specific.
- Example: jemalloc has no inline metadata!

Metadata Corruption Goals

- What might we want to achieve with heap metadata corruption?
- **Modify** arbitrary memory.
- Achieve an **overlapping allocation** (with other heap structures, stack, etc).
- Use either of those capabilities for **further control**.